

EXPERIENCE GOVERNANCE KERNEL

evo-kernel

双 Harness 经验内核 —— 工程项目开发能力

把 AI 编码 harness 从「无记忆的工具」，升级为带项目经验、带安全护栏、经验可积累、可验证的工程开发系统。

pre_llm_call → recall

on_session_end → session-end

pre_tool_call → guard

2026-08 · euly · Hermes × Claude Code · 纯文件 + git 内核

一句话定位

evo-kernel 是一个「**经验内核**」：纯文件 + git 存储的知识库，经 3 个 hook 挂到 AI 编码 harness，把每一次对话自动转化为可检索、可复用、可固化的工程经验。

260+

已验证条目
(manifest)

6 类

记忆形态
facts / playbook / skills...

3 × 2

hooks × 双 harness
Hermes / Claude Code

经验即数据

markdown + git 版本化，全程可审计

零依赖内核

单份 Node CLI (26 命令)，无外部服务

协议唯一

只认 Claude hook 契约，换 harness 只换 adapter

总体架构：一个内核，双 harness 接线

evo-kernel 经验内核

纯 markdown + git · Node CLI 26 命令 · 目录即状态机

inbox

lessons

facts

episodes

playbook

principles

skills

constraints

adapter 适配层 ~/.hermes/agent-hooks/

Hermes payload → evo 协议翻译 · 全部 fail-open (故障不阻塞主链路)

evo-recall.sh

evo-session-end.sh

evo-guard.sh

Hermes

config.yaml · hooks 段

Claude Code

settings.json · 同三件套

协议唯一 · 适配器换装

内核只认 Claude hook 契约；接入新 harness 只需写 3 个 adapter，双端共享同一份经验。

内核无状态

零依赖 Node + markdown；git 即备份，也做审计。仓库位置无关。

双写通道

会话结束双写：git 治理层（可审计）+ gbrain 语义脑（向量检索）。

三 hook：经验在对话中的实时闭环

`pre_llm_call` → `recall`

检索注入

- 每次 LLM 调用前，按当前任务语义检索经验
- 按 triggers / tags 计权，注入上下文
- 让新对话「记得」所有历史教训

`on_session_end` → `session-end`

登记 + 双写

- 会话结束自动登记 transcript 引用
- 双写 gbrain 语义脑 (put_page + timeline)
- 沉淀为可蒸馏的工程素材

`pre_tool_call` → `guard`

硬约束拦截

- 工具调用前执行硬约束规则
- 命中（如危险 `rm -rf`）即 deny 阻断
- 经验从「建议」升级为「护栏」

I1 · fail-open

任何 hook / adapter 故障都不阻塞 harness 主链路 —— 经验系统是「增强」而非「依赖」，主链路永远可跑。

目录即状态机：条目的位置 = 条目的状态

目录角色与注入资格 (I2 守恒)

inbox/	capture 暂存 + 会话登记	永不注入
lessons/	candidate 经验暂存	永不注入
facts/	语义记忆 · 事实/偏好/环境	注入
episodes/	情景记忆 · 一次任务一份	注入
playbook/	策略库 · 原子 bullet	注入
principles/	原则 · 跨域普适	注入
skills/	程序记忆 · SKILL.md 包	skill 注入
constraints/	硬约束 · guard 执行	实时拦截
archive/	退役/固化存档	不检索

状态迁移

inbox → **curate** →

facts / episodes / playbook / principles
人审唯一入库口

validated → **solidify** →

skills (程序记忆) / constraints (硬约束)
反复验证后固化

validated → **demote** →

archive (退役) / lessons (回炉)
失效条目退出注入

I3 · 人审前置 — **curate** 是唯一入库口

一条经验的完整旅程



为什么有效：条目不随会话蒸发

经验以「候选 → 验证 → 固化」递进；验证靠实战计数（helpful/harmful），差经验被审计与退役，库里长期留存的都是经实战验证的结论。

六类记忆：工程能力的沉淀载体

episodes

6

情景记忆

一次任务一份工程档案

任务级结论 / 决策 / 证据

facts

18

语义记忆

项目环境与已验证结论

按 domains 分域沉淀

playbook

90

策略库

原子操作手册

每条带 3-5 个 triggers

principles

3

原则

跨域普适开发纪律

方法论层面，适用所有项目

skills

5

程序记忆

SKILL.md 可直接执行

反复验证后固化

constraints

2

硬约束

guard 实时拦截

危险操作机器强制执行

案例特写：UCM221 雷达项目

episode · faf-legacy-gate-domain-bugs

FAF 移植发现两个静默门限 bug

俯仰门限作用在错误的量上（弧度 vs sin），方位门限是从未生效的死代码。

效果可量化

航迹覆盖 99.7% → 100.0% · 误差 0.18 m → 0.12 m

下次任何人再碰 FAF 门限，这些坑在动手前已注入。

playbook · 策略弹药库

chamber-doa-bin3-fixed-gate-rx-pair

暗室测角固定门限的 RX 配对方案

radar-doppler-bin-wraparound-unroll-first

多普勒 bin 卷绕先展开再处理

armv7-cross-compile-needs-mfpu-neon

ARMv7 交叉编译必须显式开 NEON

facts · 项目现状随手可查

交叉角数据集档案 · 仓结构重构记录 · tracker 宏定义位置 · ARM 超分辨率性能结论 —— 新会话进来项目「现状」已在场。

闭环效果：三大工程能力



。 下一次会话开始时，历史经验已在场 —— 回到「会话发生」

项目知识不随会话蒸发

每次对话结论落为持久化条目；双 harness 共享同一份库，跨工具、跨天、跨会话连续。

经验在最需要的时刻出现

recall 按 triggers 语义匹配注入；不必主动检索，相关教训动手前已在上下文。

质量有账可查

每条经验带 helpful/harmful 计数与验证等级；差经验被统计、审计、退役。

SUMMARY

经验内核，让 AI 开发越用越强

把 AI 编码 harness 从「无记忆的工具」，升级为带项目经验、带安全护栏、经验可积累、可验证的工程开发系统。

经验即数据 · git 可审计

三 hook 实时闭环

人审前置 + fail-open

六类记忆 + 硬约束护栏

evo-kernel · ~/Dev/evo-kernel · 纯文件 + git · Hermes × Claude Code